

ENCM 511: Embedded System Interfacing
Assignment 1

Group 6

Chloe Fulbrook - 30210975

Ethan Lee - 30213225

Devon Calvin - 30204532

Submitted on September 24, 2025

General Code Explanation

The peripherals are initialized inside the main function but outside the while(1) loop. The outputs on RB5, RB6, RB7 are initialized using `TRISBbits.TRISBx = 0`. The inputs on RA4, RB8 are initialized using `TRISBbits.TRISBx = 1` for the buttons. Additionally, those inputs are pulled up using `IOCPUXbits.CNPUX = 1`.

```
int main(void) {  
  
    TRISBbits.TRISB5 = 0;    // set rb5 as output (LED0)  
    TRISBbits.TRISB6 = 0;    // set rb6 as output (LED1)  
    TRISBbits.TRISB7 = 0;    // set rb7 as output (LED2)  
  
    TRISAbits.TRISA4 = 1;    // set ra4 as input (PB0)  
    TRISBbits.TRISB8 = 1;    // set rb8 as input (PB1)  
  
    IOCPUAbits.CNPUA4 = 1;    // pullup on ra4 for button  
    IOCPUBbits.CNPUB8 = 1;    // pullup on rb8 for button
```

The logic of the code uses if statements to check if the inputs are low for the button presses, and executes the specific code for each LED. The specific logic for the button presses is discussed in more depth later in this document. Our while(1) superloop begins by checking if both buttons are pressed and handles this case, then moves to check if one or the other is pressed, and makes the LEDs blink accordingly. There is also the else statement at the end to default all of the LEDs to off.

For variable datatypes, we are using `uint32_t` types for the for loop timers to ensure there is enough resolution to hold the whole number. This is also discussed in more depth later in the report.

What datatype did you choose to use to help with implementing the delays needed for the different blink rates?

The data type we used was an unsigned 32 bit integer.

Why did you need to use this type?

We used this type as the maximum range of an unsigned 16 bit integer (65536) is too small for the amount of cycles we calculated the 2 second loop would need (400000).

The calculation for this was based on that a single instruction takes 250ns (4MHz clock) and one cycle of a for loop takes around 20 instructions based on the disassembly code. The calculation below was done for the 2 second delay.

$$\frac{2s}{\frac{250ns}{1 \text{ instruction}} * 20 \text{ instructions}} = 400\,000$$

Does your system immediately turn off the LED after you release a button? Why/why not? (note, it is okay if your system does not immediately turn off the LED, but you should be able to explain why)

Yes, as each delay for loop checks if the buttons are still pressed and executes a break statement if it is released. Without this break statement, the delay loop will execute for its full length, leaving the LED on for that amount of time, which would be especially noticeable for the longer 2 second delay.

How does your PB press detection logic work?

In the infinite while loop in main, if/else logic checks the states of the buttons on each iteration of the loop. In the physical controller, we have the buttons wired so one side goes to the microcontroller pin with a pullup and the other side to ground. So, when the button is pressed, the pin should be read as a logic low (0). In the code, we check a state where both PB0 and PB1 are low to turn on LED2 constantly, then checking if one is high and one is low for the other states. Also, in each of our delay for loops there is an if statement that checks if PB0 or PB1 is logic high (1) and executes a break statement if that condition is met. This will turn the LED's off when the user releases the button